

A Survey on Vulnerabilities of Service Workers

Yeomin Jeong

Department of Computer Science and Engineering
Korea University
Seoul, Republic of Korea
ymjeong@isslab.korea.ac.kr

Junbeom Hur

Department of Computer Science and Engineering
Korea University
Seoul, Republic of Korea
jbhur@korea.ac.kr

Abstract—In a Progressive Web App (PWA), a kind of application software of the web, a service worker (SW) plays a key role as a one of the fundamental components to enhance the user's browsing experiences. For this purpose, the SW supports several features such as push notification, offline access, background code execution, etc. Since the SW provides prolific capabilities, it has been the main target to abuse by malicious attackers to deliver diverse attacks through the web applications such as cryptocurrency mining, history sniffing, phishing. In this paper, we introduce the SW's functionalities and vulnerabilities, and discuss the existing attack methodologies and their implications.

Index Terms—Service worker, Progressive Web App (PWA), web push, phishing, history sniffing

I. INTRODUCTION

Progressive Web App (PWA) [1] is a kind of a web application software. Since it provides a reliable experience to users, it is supported by many modern websites. PWA supports both features of the web app and the native app. While the web app is convenient to reach to the user, the native app can use many capabilities of the device such as camera, microphone, GPS, swipe gesture, etc [2], [3]. However, the web app does not support many capabilities like the native app, and the native app has to be built in the other languages by the platform which increases the development cost. In order to solve these limitations and enhance the users' experience, the PWA is designed to support the merits of both the web app and the native app. For example, the users can find and access PWA easily through the search engine on the Internet since the PWA is implemented as a website. PWA can use the device's resource, and can be added to the device's home screen for the convenient access, which looks similar to the native app. Also, PWA can be used in the offline or slow network environments because of the support of a service worker (SW).

A SW [4] is an essential component of PWA to enhance the users' browsing experience by using a cache, synchronizing in the background, and being executed in a different thread from the browser. The SW uses a cache to store the recently used or static data to support fast loading and offline access. Moreover, since the SW is executed in its own thread independently, it can execute code in the background even when the browser is close, such as receiving the push notification from the remote server and background synchronizing to support a continuous browsing experience. With these features, the SW can provide prolific services regardless of the browser's network condition.

However, several studies have disclosed vulnerabilities of the SW, and proposed the attacks on it by exploiting them. In this paper, we first introduce the existing attacks which utilize the push notification to lure the users to link to the malicious site [5], or to track the users' location [6]. Second, we introduce the attacks that exploit the SW by utilizing its features. These attacks include affecting the SW from the DOM [6], [7], injecting the attacker's code to the benign SW [8], and installing the attacker's SW on the victim's browser [9]. At last, we introduce the browsing history sniffing attacks [5], [10] which use the SW's existence to determine whether the users have accessed target sites or not.

The main goal of this paper is to introduce the existing studies on the PWA attacks, and discuss their implications for the new researchers who dive into this area.

II. BACKGROUND

In this section, we introduce the SW and address its four main capabilities for enhancing the users' browsing experiences.

A. Service Worker

The SW is an application software delivered through the web. Specifically, it is implemented as a JavaScript file that is an essential component of PWA, and it works by event-based operations. For safety, the SW is only allowed to be used in the PWA sites that are built in an HTTPS environment. When users access the PWA, the users' browser registers the site's SW by downloading the SW from the site. In this process, the scope of the SW is defined, which only can be below the respective path [11]. Then, the service worker can access the resources within its scope following the life-cycle [12] shown in Figure 1. At first, the *install* event occurs if the registration process is done, and the browser's cache can be initialized and filled with the resources which are already designated in the SW script. If there is a past version of the SW, it is removed during this process. Next, if there is not any error in the *install* event, the *activate* event occurs, and the SW becomes *activated* state. In this state, the SW can control clients and handle functional events like *fetch*, *push* and *sync*. If the SW does not receive any functional events, the SW is changed into a *terminated* state, and does not execute the code in the script. However, if it receives another event later, it becomes *activated* state again and can handle the event.

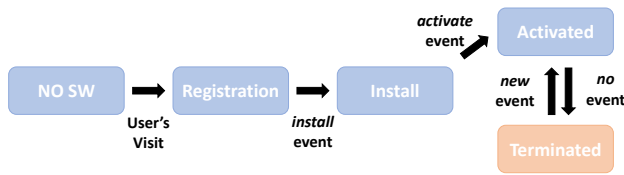


Fig. 1. Life-cycle of a service worker.

During the life-cycle, the SW uses the browser's cache to support fast loading and offline usage, executed in a different thread with the browser's thread. It thus can support displaying push notifications and background code execution even when the browser is closed.

Fast Loading and Offline Usage. In the *install* event, the SW stores the static resources of the site, such as image, HTML file, JavaScript files in the browser's cache. When users access the website, the website requests the resources from the remote server of the website. In PWA, the SW intercepts the request and handles it. Specifically, if there is a requested resource in the cache, the SW responds with the cached data instead of fetching it from the remote server. If there are no related resources in the cache, the SW brings the requested resource from the remote server. Moreover, the SW can store the request and response in the cache to use them for the same request later. With the cached data, the SW can provide fast loading of the site to users.

As mentioned above, the SW can store the static resources in the browser's cache. In addition to the fast loading, the cached resources can be used to support offline usage. Even when the network is not available (such as temporarily disconnected), the SW displays the site to the users using the cached resources. Therefore, the users may not be hindered by the network condition until the network connection is recovered.

Background Code Execution and Push Notification. As the SW is executed in a different thread from the browser's thread, it can execute the code in the background such as for the events handling. Among the events, the *push* event is one of the most frequently used events in the SW. After users subscribe to the site to receive the push notification from the site, the users' browser sends subscription information to the site including the *endpointURL*. The *endpointURL* is the URL address of the SW, indicating the destination of the push message. Then, the SW can receive the push message from the site's server, and display the push notification to the users instantly. This push notification can be displayed even when the browser is closed. Thus, the push notification can be relayed to the users at any time as soon as the SW receives the push message.

III. ATTACKS EXPLOITING THE SERVICE WORKER

In this section, we present the existing studies of exploiting the SW: attacks utilizing the push notification, exploiting the SW's features, and extracting users' browsing history.

A. Utilizing Push Notification

As mentioned in Section II-A, receiving and displaying the push notification is one of the most essential features of the SW. When the website's remote server sends a push message to the subscribed user, the SW receives the push message and displays the push notification on the users' browser. In order to send the push message to the corresponding subscribed users, the remote server uses *endpointURL* to find the correct users since each SW has a distinct endpoint. However, there is a danger of abusing the SW's *endpointURL* by an attacker to send a crafted push notification, aiming to lure the users to click it which leads the users to a malicious site, such as a phishing site.

In 2018, Lee et al. [5] analyzed some vulnerabilities of the SW. They introduced a risk of third-party push services, which are frequently used in many of the modern websites up to almost 70% [5] to support the push notification easily. The developer of the site can send a push notification to the users who subscribed to their site from the third-party service by using the source code of the third-party's. These third-party service also allows delivery of a push notification on HTTP sites. Since the HTTP site does not have the SW to use for receiving and displaying the push notification to the users, they make their users subscribe to the subdomain of the third-party service which is built in HTTPS. However, since the connection between the third-party service and the HTTP site is established through the HTTP protocol, the attacker can eavesdrop on the data including the *endpointURL*, especially when the users' browser sends subscription information to the third-party service. It can further allow the attacker to send the crafted push message to the users.

Chinprutthiwong et al. [6] also presented an attack that can hijack the push subscription information from the benign SW to acquire the users' location from the push notification in 2021. They use the feature of some third-party push service, which can get the users' location (longitude and latitude) if the users click a push notification. In this attack, the attacker injects malicious codes to the HTML of the site to replace the *appId* of the subscription from the benign site's *appId* with the attacker's *appId*. Then, the attacker can hijack the push subscription of the third-party service, and use it to track the users' location.

In addition, several recent studies [5], [9] proposed attacks exploiting the push notification as an attack vector to keep the SW awake to carry on malicious works such as cryptocurrency mining, since the push notification is one of the events that can be handled in the SW.

B. Exploiting the SW by Utilizing its Features.

Since Lee et al.'s study [5], several studies have presented attacks targeting the SW's features. They are designed to affect the SW from the document object model (DOM), inject the attacker's script to the benign SW, and install the attacker's SW.

Affecting the SW from the DOM. The DOM is the data representation of the objects that comprise the structure and

content of a document on the web [13]. The SW cannot access the DOM directly whereas the DOM can access the SW. Thus, tainting the DOM can make the SW vulnerable. In the previous studies [6], [7], IndexedDB and Cache API were used as attack vectors to exploit the SW since they are used in both the SW and the DOM. In the attacks, the attacker first conducts the XSS attack to inject the attacker's script into the DOM. Then, the injected script can modify the contents of the IndexedDB or browser's cache using Cache API, which is used in the SW too. Through these attacks, the attacker can modify the URL in the IndexedDB to inject malicious codes into the SW, or can remove the security header in the cache to break the security of the SW.

Injecting Attacker's Script to Benign SW. In 2020, Chinprutthiwong et al. [8] presented an attack called 'SW-XSS', which can inject the attacker's script into the benign SW. In this attack, the website's benign SW has to use the URL parameter to fetch the data in its SW as a prerequisite. The attacker can make a crafted URL to inject the URL of the attacker's script into the benign SW to exploit it. When the victim clicks the attacker's crafted URL, the URL's parameter is used in the SW's functions such as *importScripts()*, which injects the script using the URL as its parameter. Then, the attacker can control the benign SW with his script such as for executing the malicious code.

Installing the Attacker's SW. Papadopoulos et al. [9] introduced the web bot called 'MarioNet' that can exploit the victim's computer resources by installing the attacker's SW. In this attack, the attacker makes the victim visit a malicious site such as the phishing site or a vulnerable site whose SW is corrupted by the attacker. Phishing advertisements or crafted URLs can be methods. If the victim visits the malicious site, the attacker's SW is installed in the victim's browser without any notification. After that, the attacker can abuse the victim's computer resources using the SW in the background such as conducting crypto-currency mining, generating darkweb, and conducting DDOS attacks.

C. Sniffing Users' Browsing History

As the SW is installed after the users' first visit to the site, Karami et al. [10] used the SW as a criterion to determine whether the users' have visited the site before. They probed whether the target sites' SW is installed in the users' browser by making the users visit the target sites again. If the users have visited the target sites before, the time interval to connect to the site is shorter than the first visit since the SW uses the browser's cache to load the contents. On the other hand, if the users have never visited the target sites before, the SW has to load the contents from the remote server. Thus, the connection time increases. With this information, the attacker can conduct a browsing history attack on the users. However, this attack is valid only a single time, since the SW will also be installed after the users' first visit to the target sites which occurred during the attack regardless of a past visit. Similarly, Lee et al. [5] also used the SW existence to sniff the users' browsing history by checking whether the *onload* event is generated.

The *onload* event occurs if the SW is registered by the users' visit before.

IV. CONCLUSION

PWA is a useful technology for a modern website that supports both advantages of the web app and the native app. PWA can support the easy access and usage of many capabilities to enhance the users' browsing experiences. The service worker (SW), the essential component of the PWA, provides fast loading and offline usage through cache, and provides background code execution and push notification. In this paper, we introduced the SW including its life-cycle and capabilities. Then, we discussed the existing attacks which target the SW by utilizing push notifications, exploiting its features, and extracting users' browsing history. Finding more diverse vulnerabilities of the SW and attack methods, and designing effective countermeasures will be an important future work for securing the current web systems.

ACKNOWLEDGMENT

This work was supported by IITP grant funded by the MSIT, Korea(No.2019-0-00533, No.2022-0-00411, IITP-2022-2021-0-01810) and Basic Science Research Program through the National Research Foundation funded by the Ministry of Education, Korea(NRF-2021R1A6A1A13044830).

REFERENCES

- [1] Web.Dev., "What are progressive web apps." 2020. [Online]. Available: <https://web.dev/what-are-pwas/>
- [2] CSSChopper., "Web app vs progressive web app: Why are they different from each other?" 2022. [Online]. Available: <https://www.csschopper.com/blog/web-app-vs-progressive-web-app/>
- [3] magestore., "Pwa vs native app and how to choose between them." 2021. [Online]. Available: <https://www.magestore.com/blog/pwa-vs-native-app-and-how-to-choose-between-them/>
- [4] mdn web docs., "Service worker api." 2018. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API
- [5] J. Lee, H. Kim, J. Park, I. Shin, and S. Son, "Pride and prejudice in progressive web apps: Abusing native app-like features in web applications," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1731–1746.
- [6] P. Chinprutthiwong, R. Vardhan, G. Yang, Y. Zhang, and G. Gu, "The service worker hiding in your browser: The next web attack target?" in *24th International Symposium on Research in Attacks, Intrusions and Defenses*, 2021, pp. 312–323.
- [7] M. Squarcina, S. Calzavara, and M. Maffei, "The remote on the local: exacerbating web attacks via service workers caches," in *2021 IEEE Security and Privacy Workshops (SPW)*. IEEE, 2021, pp. 432–443.
- [8] P. Chinprutthiwong, R. Vardhan, G. Yang, and G. Gu, "Security study of service worker cross-site scripting," in *Annual Computer Security Applications Conference*, 2020, pp. 643–654.
- [9] P. Papadopoulos, P. Ilia, M. Polychronakis, E. P. Markatos, S. Ioannidis, and G. Vasiladis, "Master of web puppets: Abusing web browsers for persistent and stealthy computation," *arXiv preprint arXiv:1810.00464*, 2018.
- [10] S. Karami, P. Ilia, and J. Polakis, "Awakening the web's sleeper agents: Misusing service workers for privacy leakage," in *Network and Distributed System Security Symposium*, 2021.
- [11] mittl medien., "The scope of the service worker (progressive web app - pwa)." 2020. [Online]. Available: <https://mittl-medien.de/progressive-web-app/scope-service-worker>
- [12] Web.Dev., "The service worker lifecycle." 2016. [Online]. Available: <https://web.dev/service-worker-lifecycle/>
- [13] mdn web docs., "Introduction to the dom." 2014. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction